

Waterfall model

Requirements Engineering

1. Feasibility Study

The technology does exist to create this system. (Classes, objects, databases, interface, usw.)

2. Requirements elicitation

- a. We have examined past systems such as Leopard Web. From this, the requirements are...

- i. Courses

- 1. Database must contain:

- a. CRN
 - b. Course names
 - c. Time
 - d. Instructor

- 2. Must contain multiple semesters, print out of schedules and preferences.

- ii. Users

- 1. Must be able to support 3 classes of users:

- a. Admin – can see everything and can add/delete/edit courses, users, and schedules.
 - b. Instructor – can see available courses and their own rosters.
 - c. Student – to register for and see available courses as well as their own schedule.

- 2. Must support at least 100 students, 10 instructors, and 1 administrator.

- iii. Must work and undergo major testing.

- b. Classes:

- i. User:

- 1. Base class must have First name, last name, and ID. Must also have a set function for each attribute and a way to print all info.

- 2. Student sub class

- a. Must be able to search for and add/drop courses. Must also be able to print their schedule.

- 3. Instructor sub class

- a. Must be able to print their schedule and class list as well as searching for courses.

- 4. Admin sub class

- a. Must be able to add/remove courses and users to the system, add/remove students from courses, and search/print rosters and courses.

3. Requirements validation

Design and Implementation

- a. Architectural design
 - i. User class and 3 subclasses of user (Student, Admin, and Instructor)
 1. Allow them to do everything as specified in requirements section.
 - ii. Database of classes, users, CRNs, usw.
 - iii. Interface that allows a user to login and to perform all functions the user should be able to.
 - b. Interface design
 - i. Login window
 - ii. Clickable search window to find courses.
 - iii. The ability to add/remove courses/students for users of the applicable classes.
 - c. Component
 - i. Implement User class and all 3 subclasses.
 - ii. Make sure all users can do all tasks required and cannot access anything they shouldn't be able to.
 - iii. Add database of courses and users and integrate them with the classes.
 - iv. Implement components of the GUI
 - d. Database design –
 - i. Class Table
 1. Separated by semester.
 2. Contains CRN, time, and instructor.
 - ii. User Tabel
 1. Contains login info, type of user, schedule, First Name, Last Name, and ID.
4. Component testing
 - a. test the individual components as they are being built.
 5. System testing
 - a. test the system after integration.
 6. Acceptance testing
 - a. test the system with “real” data.
 7. Software Evolution
 - a. Modify the system based on changing user needs over time or any bugs found.

Incremental Model

1st version:

- Create the User base class and all subclasses.
 - o Users should be able to:

- Base class must have First name, last name, and ID. Must also have a set function for each attribute and a way to print all info.
- Student sub class
 - Call the functions to be able to search for and add/drop courses. Must also be able to print their schedule.
- Instructor sub class
 - Call the functions to be able to print their schedule and class list as well as searching for courses.
- Admin sub class
 - Call the functions to be able to add/remove courses and users to the system, add/remove students from courses, and search/print rosters and courses.

2nd version:

- Database should be implemented.
 - Classes should be integrated with database to allow methods to work as intended
- Begin crude GUI
 - Basic text functions
 - Class printouts
 - Course search
 - Usw

3rd version:

- Polish GUI
 - Easier to use interface.
 - User login functions.
 - Better accessibility and readability.
- Make sure all users can access their required functions.

Final version:

- Fix any bugs that may be prevalent from previous versions.
- Add more semesters, full datasets, and extend user and course datasets.